Ch. 2 Cheat Sheet

Algorithms, Recursion, Strategies

Problem vs. algorithm

A **problem** specifies inputs → outputs (*what*). An **algorithm** is a procedure that solves it (*how*). One problem usually has many correct algorithms; you pick by correctness, speed, memory, simplicity.

What makes an algorithm "good"

1. **Correct** on all inputs (including edges).
2. **Fast enough** for input sizes you'll see.
3. **Reasonable memory**.
4. **Simple enough** to implement, test, maintain.

Recursion anatomy

Every recursive function must have:

- A **base case** returning without recursing.
- A **recursive case** that calls itself on a *strictly smaller* input.

If the recursive call isn't smaller or doesn't converge to the base, you stack-overflow.

```
int factorial(int n) {
    if (n <= 1) return 1;         // base
    return n * factorial(n - 1); //
        recurse
}
```

Binary search (recursive)

```
int bsearch(const vector<int>& v,
            int lo, int hi, int t) {
    if (lo > hi) return -1;       // base
        : miss
    int m = lo + (hi - lo) / 2;
    if (v[m] == t) return m;
    if (v[m] <  t) return bsearch(v, m
        +1, hi, t);
    return                bsearch(v, lo,
        m-1, t);
}
```

Precondition: v sorted. **Cost:** $O(\log n)$.

Recursion gotchas

- Missing base → infinite recursion / stack overflow.
- Recursive call not smaller → same.
- **Overlapping subproblems** (naïve Fib) → exponential blowup. Memoize or iterate.
- Deep stacks cost memory; iterative form may be required for huge n.

Heuristic vs. greedy vs. DP

Heuristic	Good-enough rule; no optimality guarantee.
Greedy	Locally optimal choice each step; optimal *only* when greedy-choice property holds.
DP	Optimal by exploring smaller subproblems, storing results; needs optimal substructure + overlapping subproblems.

Greedy: when it works

Use greedy iff the problem has the **greedy-choice property**: the locally optimal choice is always part of *some* globally optimal solution. Classic wins:

- Fractional knapsack (sort by value/weight, take).
- Activity selection (earliest finish time).
- Coin change with "canonical" coin systems (US coins OK; arbitrary systems *not* OK).

Classic fails:

- 0/1 knapsack (greedy on ratio is wrong).
- Coin change with $\{1, 3, 4\}$, target 6: greedy picks $4+1+1$, optimum is $3+3$.

DP: the template

1. Identify **state**: what parameters uniquely determine a subproblem?
2. Write the **recurrence**: optimal for state s in terms of smaller states.
3. Identify **base cases**.
4. Choose **order** to fill the table (bottom-up) or use memoization (top-down).
5. Space-optimize if only the last row/col is needed.

Signals a problem is DP: asks for optimum/count/exists; recurrence reuses same subproblem many times.

Recognizing the strategy

"Find optimum" + choice-at-each-step, greedy works	**Greedy**
"Find optimum" + overlapping subproblems	**DP**
Too expensive to solve exactly, need "good enough"	**Heuristic**
Natural smaller-subproblem structure, each solved once	**Div & Conq**

Data privacy & ethics – the two levers

- **Collection.** Only gather what you need; de-identify when possible; plan retention.
- **Release.** Aggregate, suppress small cells, use differential privacy or k-anonymity; assume auxiliary data exists.

Re-identification via auxiliary data is the most common failure mode (Netflix, AOL, NYC taxi logs).

Ethics checklist (CS 300 flavor)

- **Harm.** Who might this code hurt?
- **Consent.** Did users agree to *this* use?
- **Transparency.** Can I explain the decision logic?
- **Fairness.** Does it disparately affect a group?
- **Reproducibility.** Could someone rerun this and audit?

Top gotchas

- Claiming greedy is optimal without checking the greedy-choice property.
- Writing DP without defining the state first.
- Naïve recursion on a problem with overlapping subproblems (Fibonacci trap).
- Using a heuristic result as if it were optimal.